

Practical Session 4

Task 1

Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/wait_waitpid.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/wait_waitpid.c -o build/wait_waitpid
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/wait_waitpid
Parent Process Started
Parent PID: 4817

Child 1: PID = 4818, PPID = 4817

Parent created two children:
Child 1 PID: 4818
Child 2 PID: 4819

Using wait()...
Child 2: PID = 4819, PPID = 4817
Child 1 completed.
wait() returned PID: 4818
Child exited with status: 10

Using waitpid() for Child 2...
Child 2 completed.
waitpid() returned PID: 4819
Child 2 exited with status: 20

Parent process exiting.
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t child1, child2;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    child1 = fork();

    if (child1 < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0)
    {
        printf("Child 1: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(2);
        printf("Child 1 completed.\n");
        exit(10);
    }

    child2 = fork();

    if (child2 < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child2 == 0)
    {
        printf("Child 2: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(4);
        printf("Child 2 completed.\n");
        exit(20);
    }

    printf("\nParent created two children:\n");
}
```

```
GNU nano 8.7.1 src/wait_waitpid.c *
}

if (child2 == 0)
{
    printf("Child 2: PID = %d, PPID = %d\n", getpid(), getppid());
    sleep(4);
    printf("Child 2 completed.\n");
    exit(20);
}

printf("\nParent created two children:\n");
printf("Child 1 PID: %d\n", child1);
printf("Child 2 PID: %d\n\n", child2);

printf("Using wait()...\n");

int status;

pid_t completed = wait(&status);

printf("wait() returned PID: %d\n", completed);

if (WIFEXITED(status))
{
    printf("Child exited with status: %d\n",
           WEXITSTATUS(status));
}

printf("\nUsing waitpid() for Child 2...\n");

waitpid(child2, &status, 0);

printf("waitpid() returned PID: %d\n", child2);

if (WIFEXITED(status))
{
    printf("Child 2 exited with status: %d\n",
           WEXITSTATUS(status));
}

printf("\nParent process exiting.\n");

return 0;
}
```

Task 2

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
j_rakshitha@RAKSHITHA:~/ossproject$ nano src/zombie_demo.c
j_rakshitha@RAKSHITHA:~/ossproject$ gcc -Wall -Wextra -g src/zombie_demo.c -o build/zombie_demo
j_rakshitha@RAKSHITHA:~/ossproject$ ./build/zombie_demo
Zombie Process Demonstration
Parent PID: 4865

Parent created child with PID: 4866
Parent will NOT call wait() immediately.
Waiting for 15 seconds so the zombie can be observed...

Child Process
Child PID: 4866
Child is exiting now...
Parent is now calling wait()...
Child has been reaped.
Zombie process eliminated.
j_rakshitha@RAKSHITHA:~/ossproject$ |
```

```
j_rakshitha@RAKSHITHA:~$ ps -eo pid,ppid,state,comm | grep zombie_demo
4865      941 S zombie_demo
4866      4865 Z zombie_demo
j_rakshitha@RAKSHITHA:~$ ps -eo pid,ppid,state,comm | grep zombie_demo
j_rakshitha@RAKSHITHA:~$
```

```
GNU nano 8.7.1 src/zombie_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Zombie Process Demonstration\n");
    printf("Parent PID: %d\n\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Child is exiting now...\n");

        exit(0);
    }
    else
    {
        printf("Parent created child with PID: %d\n", pid);
        printf("Parent will NOT call wait() immediately.\n");
        printf("Waiting for 15 seconds so the zombie can be observed...\n\n");

        sleep(15);

        printf("Parent is now calling wait()...\n");

        wait(NULL);

        printf("Child has been reaped.\n");
        printf("Zombie process eliminated.\n");
    }

    return 0;
}
```